Elective course (1)
NEURAL NETWORK AND DEEP LEARNING

Third year computer science

chapter 4

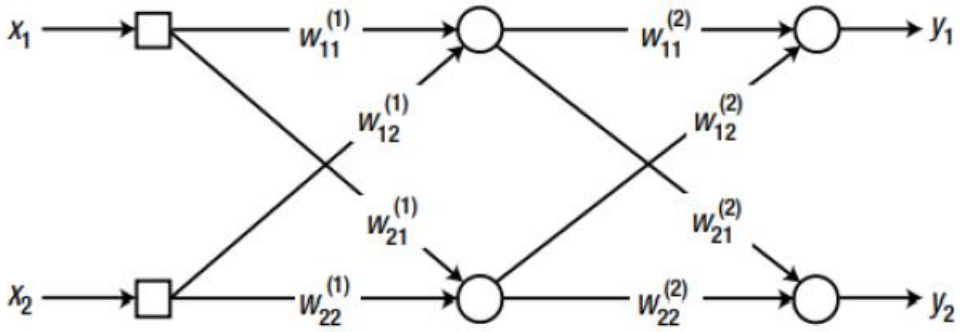
Dr. Menas Ebrahim Eissa

Training of Multi Layer Neural Network

- In 1986, the introduction of the back-propagation algorithm finally solved the training problem of the multi-layer neural network.
- The significance of the back propagation algorithm was that it provided a systematic method to determine the error of the hidden nodes.
- Once the hidden layer errors are determined, the delta rule is applied to adjust the weights.
- The input data of the neural network travels through the input layer, hidden layer, and output layer.
- In contrast, in the back-propagation algorithm, the output error starts from the output layer and moves backward until it reaches the right next hidden layer to the input layer.
- This process is called back propagation, as it resembles an output error propagating backward.
- Even in back-propagation, the signal still flows through the connecting lines and the weights are multiplied.
- The only difference is that the input and output signals flow in opposite directions.

Training of Multi Layer Neural Network

Consider a neural network that consists of two nodes for both the input and output and a hidden layer, which has two nodes as well. We will omit the bias for convenience.



To obtain the output error, we first need the neural network's output from the input data. Let's try. As the example network has a single hidden layer, we need two input data manipulations before the output calculation is processed. First, the weighted sum of the hidden node is calculated as:

$$\begin{bmatrix} v_1^{(1)} \\ v_2^{(1)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\triangleq W_1 x$$

Training of Multi Layer Neural Network

When we put this weighted sum, into the activation function, we obtain the output from the hidden nodes.

$$\begin{bmatrix} y_1^{(1)} \\ y_2^{(1)} \end{bmatrix} = \begin{bmatrix} \varphi\left(v_1^{(1)}\right) \\ \varphi\left(v_2^{(1)}\right) \end{bmatrix}$$

Where $y_1^{(1)}$ and $y_2^{(1)}$ are outputs from the corresponding hidden nodes. In a similar manner, the weighted sum of the output nodes is calculated as: :

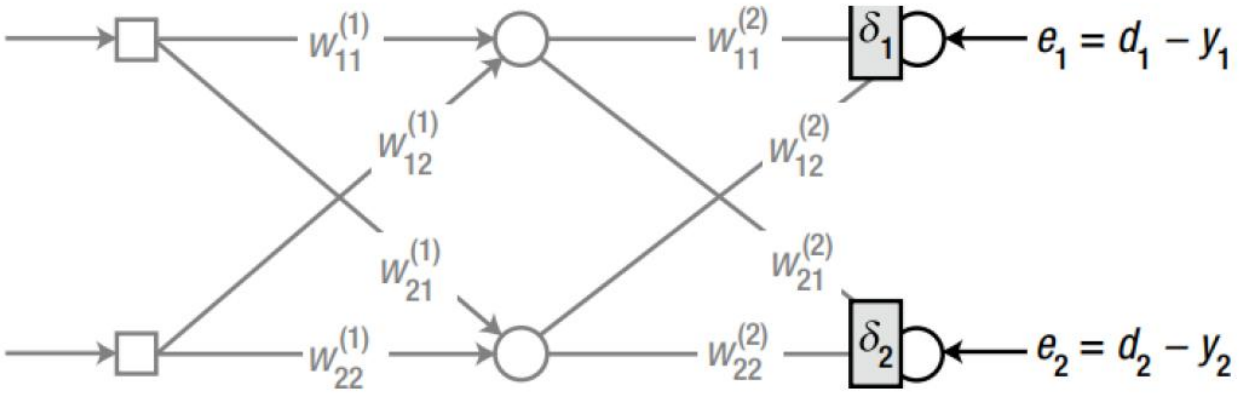
$$\begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} \end{bmatrix} \begin{bmatrix} y_1^{(1)} \\ y_2^{(1)} \end{bmatrix} \triangleq W_2\, y^{(1)}$$

As we put this weighted sum into the activation function, the neural network yields the output.

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} \varphi(v_1) \\ \varphi(v_2) \end{bmatrix}$$

Training of Multi Layer Neural Network

Now, we will train the neural network using the back-propagation algorithm. The first thing to calculate is delta, δ, of each node. You may ask, "Is this delta the one from the delta rule?" It is!



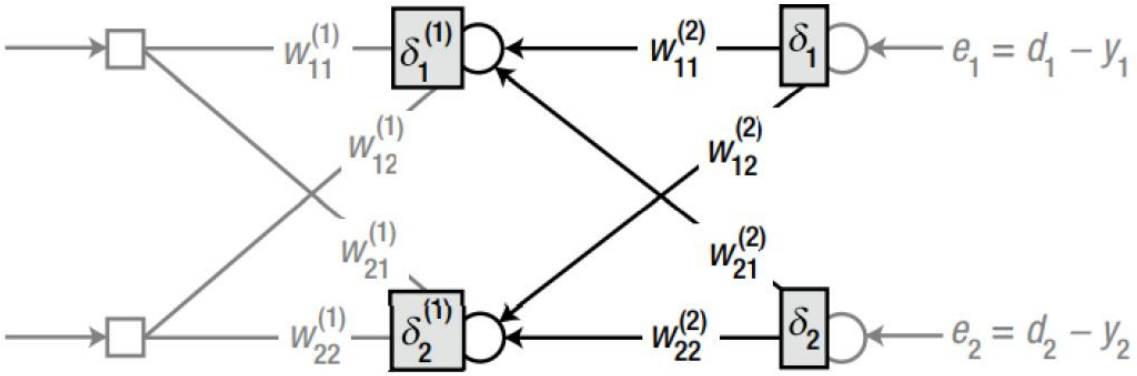
In the back-propagation algorithm, the delta of the output node is defined identically to the delta rule of the "Generalized Delta Rule" section in Chapter 2, as follows:

$$e_1 = d_1 - y_1$$
$$\delta_1 = \varphi'(v_1)e_1$$

$$e_2 = d_2 - y_2$$
$$\delta_2 = \varphi'(v_2)e_2$$

Training of Multi Layer Neural Network

where $\varphi(.)$ is the derivative of the activation function of the output node, yi is the output from the output node, di is the correct output from the training data, and vi is the weighted sum of the corresponding node. Since we have the delta for every output node, let's proceed leftward to the hidden nodes and calculate the delta. Again, unnecessary connections are dimmed out for convenience.



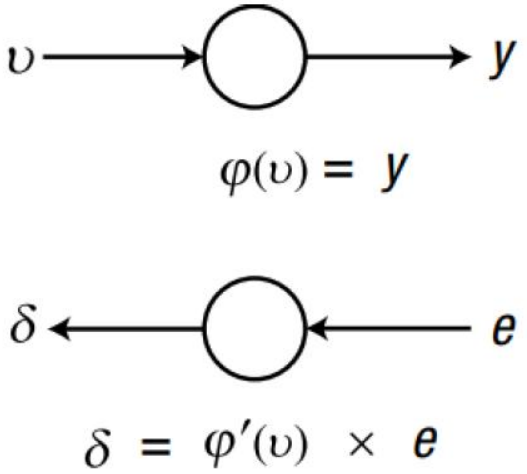
As addressed at the beginning of the chapter, the issue of the hidden node is how to define the error. In the back-propagation algorithm, the error of the node is defined as the weighted sum of the back-propagated deltas from the layer on the immediate right (in this case, the output layer). Once the error is obtained, the calculation of the delta from the node is the same previous. This process can be expressed as follows:

Training of Multi Layer Neural Network

$$e_1^{(1)} = w_{11}^{(2)}\delta_1 + w_{21}^{(2)}\delta_2$$
$$\delta_1^{(1)} = \varphi'\left(v_1^{(1)}\right)e_1^{(1)}$$

$$e_2^{(1)} = w_{12}^{(2)}\delta_1 + w_{22}^{(2)}\delta_2$$
$$\delta_2^{(1)} = \varphi'\left(v_2^{(1)}\right)e_2^{(1)}$$

where $v_1^{(1)}$ and $v_2^{(1)}$ are the weight sums of the forward signals at the respective nodes. It is noticeable from this equation that the forward and backward processes are identically applied to the hidden nodes as well as the output nodes. This implies that the output and hidden nodes experience the same backward process. The only difference is the error calculation



Training of Multi Layer Neural Network

In summary, the error of the hidden node is calculated as the backward weighted sum of the delta, and the delta of the node is the product of the error and the derivative of the activation function. This process begins at the output layer and repeats for all hidden layers. This pretty much explains what the back-propagation algorithm is about. The two error calculation formulas are combined in a matrix equation as follows:

$$\begin{bmatrix} e_1^{(1)} \\ e_2^{(1)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(2)} & w_{21}^{(2)} \\ w_{12}^{(2)} & w_{22}^{(2)} \end{bmatrix} \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix}$$

Compare this equation with the neural network output of Equation 3.2. The matrix of Equation 3.5 is the result of transpose of the weight matrix, W, of Equation 3.2.2 Therefore, Equation 3.5 can be rewritten as:

$$\begin{bmatrix} e_1^{(1)} \\ e_2^{(1)} \end{bmatrix} = W_2^T \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix}$$

Training of Multi Layer Neural Network

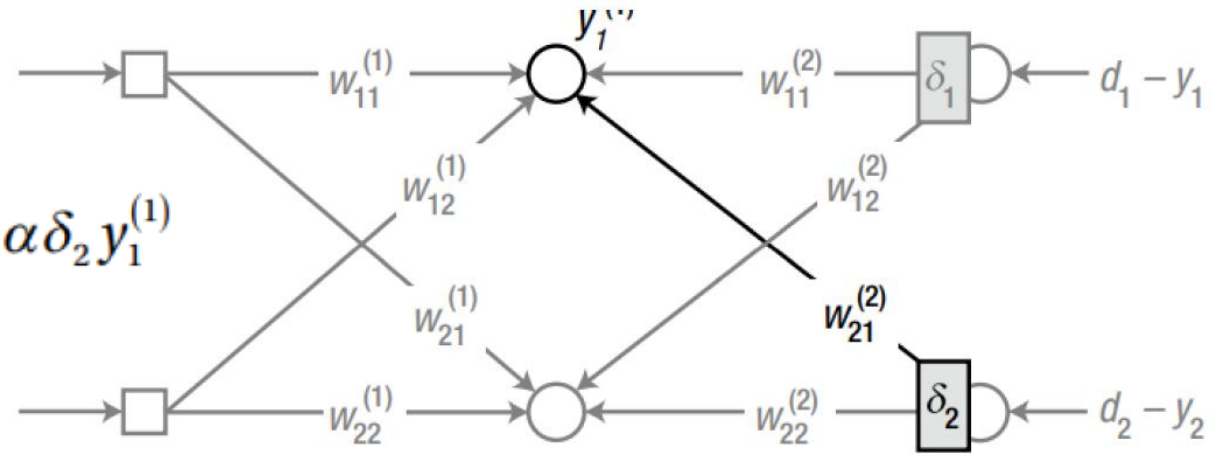
This equation indicates that we can obtain the error as the product of the transposed weight matrix and delta vector. This very useful attribute allows an easier implementation of the algorithm. If we have additional hidden layers, we will just repeat the same backward process for each hidden layer and calculate all the deltas. Once all the deltas have been calculated, we will be ready to train the neural network. Just use the following equation to adjust the weights of the respective layers.

$$\Delta w_{ij} = \alpha \delta_i x_j$$

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

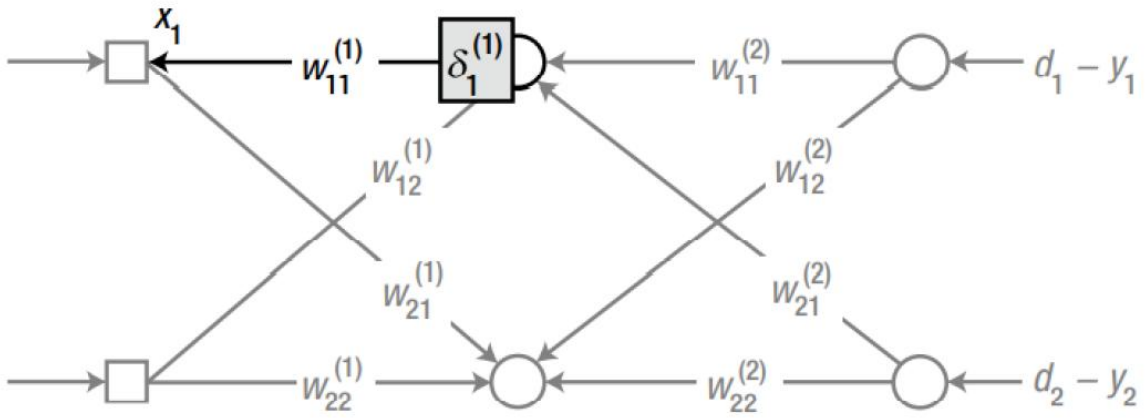
where xj is the input signal for the corresponding weight. The weight $w_{21}^{(2)}$ can be adjusted as:

$$w_{21}^{(2)} \leftarrow w_{21}^{(2)} + \alpha \delta_2 y_1^{(1)}$$



Training of Multi Layer Neural Network

where $y_1^{(1)}$ is the output of the first hidden node. Here is another example. The weight $w_{11}^{(1)}$ is adjusted using as:



$$w_{11}^{(1)} \leftarrow w_{11}^{(1)} + \alpha \delta_1^{(1)} x_1$$

where x1 is the output of the first input node, i.e., the first input of the neural network.

Training of Multi Layer Neural Network

Let's organize the process to train the neural network using the back-propagation algorithm.

1. Initialize the weights with adequate values.
2. Enter the input from the training data {input, correct output} and obtain the neural network's output. Calculate the error of the output to the correct output and the delta, δ, of the output nodes.

$$e = d - y$$
$$\delta = \varphi'(v)e$$

3. Propagate the output node delta, δ, backward, and calculate the deltas of the immediate next (left) nodes.

$$e^{(k)} = W^T\delta$$
$$\delta^{(k)} = \varphi'\left(v^{(k)}\right)e^{(k)}$$

Training of Multi Layer Neural Network

4. Repeat Step 3 until it reaches the hidden layer that is on the immediate right of the input layer.
5. Adjust the weights according to the following learning rule.

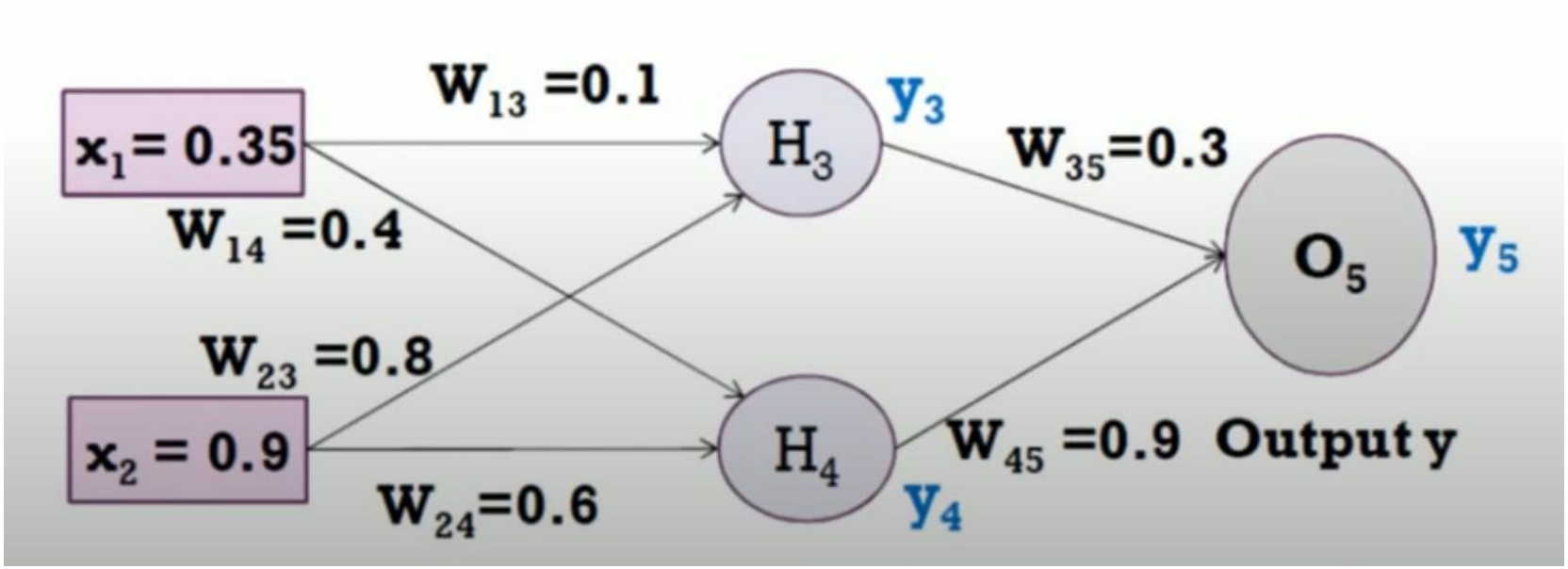
$$\Delta w_{ij} = \alpha \delta_i x_j$$
$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

6. Repeat Steps 2-5 for every training data point.
7. Repeat Steps 2-6 until the neural network is properly trained.

Other than Steps 3 and 4, in which the output delta propagates backward to obtain the hidden node delta, this process is basically the same as that of the delta rule, which was previously discussed. Although this example has only one hidden layer, the back-propagation algorithm is applicable for training many hidden layers. Just repeat Step 3 of the previous algorithm for each hidden layer.

Example

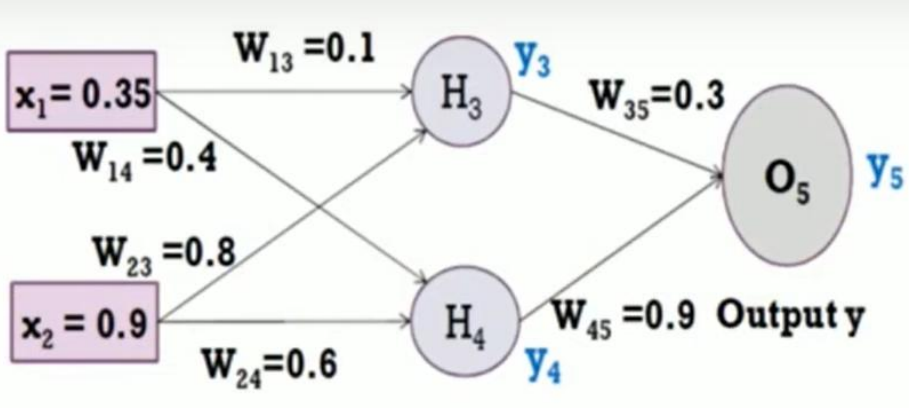
Assume that the neurons have a sigmoid activation function, perform a forward pass and a backward pass on the network. Assume that the actual output y is 0.5 and the learning rate is 1. perform another forward pass



- Forward pass:

Compute output for y3, y4, and y5

$$a_j = \sum_j (w_{i,j} * x_i) \qquad yj = F(aj) = \frac{1}{1 + e^{-a_j}}$$



$a_1 = (w_{13} * x_1) + (w_{23} * x_2)$
$= (0.1 * 0.35) + (0.8 * 0.9) = 0.755$
$y_3 = f(a_1) = 1/ (1 + e^{-0.755}) = 0.68$

$a_2 = (w_{14} * x_1) + (w_{24} * x_2)$
$= (0.4 * 0.35) + (0.6 * 0.9) = 0.68$
$y_4 = f(a_2) = 1/ (1 + e^{-0.68}) = 0.6637$

$a_3 = (w_{35} * y_3) + (w_{45} * y_4)$
$= (0.3 * 0.68) + (0.9 * 0.6637) = 0.801$
$y_5 = f(a_3) = 1/ (1 + e^{-0.801}) = 0.69$ (Network Output)

$$\text{Error} = y_{target} - y_5 = -0.19$$

Backpropagation equations

- Each weight changed by:

$$\Delta w_{ij} = \eta \delta_j o_i$$

$$\delta_j = o_i(1 - o_i)\left(t_j - o_i\right)_{\eta} \quad if\ j\ is\ an\ output\ unit$$

$$\delta_j = o_i(1 - o_i)\sum_k \delta_k w_{kj} \quad if\ j\ is\ a\ hidden\ unit$$

Where η is a constant called the learning rate.

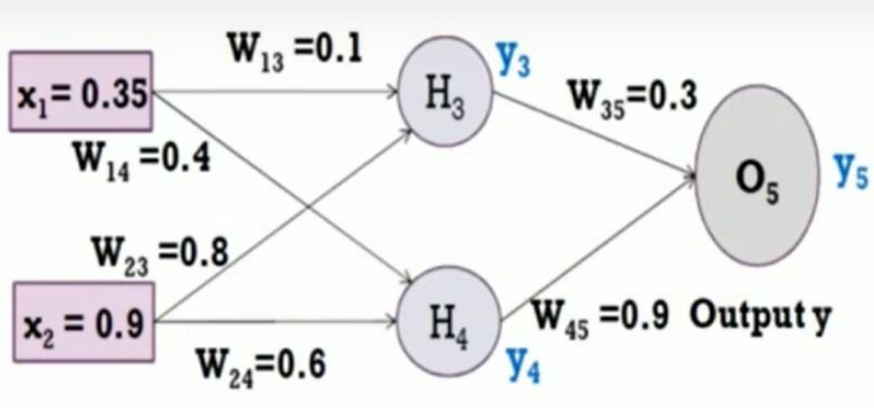
t_j Is the correct output for unit j

δ_j Is the error measure for unit j

$$\Delta w_{ij} = \eta \delta_j o_i$$

$$\delta_j = o_i(1 - o_i)(t_j - o_i) \quad \textit{if } j \textit{ is an output unit}$$

$$\delta_j = o_i(1 - o_i) \sum_k \delta_k w_{kj} \quad \textit{if } j \textit{ is a hidden unit}$$



Backward Pass: Compute δ3, δ4 and δ5.

For output unit:

$\delta_5 = y(1-y)\,(y_{target} - y)$

$= 0.69*(1-0.69)*(0.5-0.69) = -0.0406$

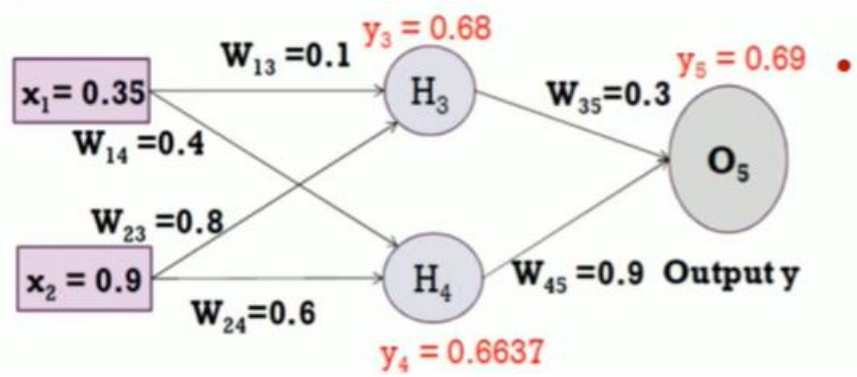
For hidden unit:

$\delta_3 = y_3(1-y_3)\, w_{35} * \delta_5$

$= 0.68*(1 - 0.68)*(0.3 * -0.0406) = -0.00265$

$\delta_4 = y_4(1-y_4) w_{45} * \delta_5$

$= 0.6637*(1 - 0.6637)* (0.9 * -0.0406) = -0.0082$



$$\Delta w_{45} = \eta\, \delta_5\, y_4 = 1 * -0.0406 * 0.6637 = -0.0269$$

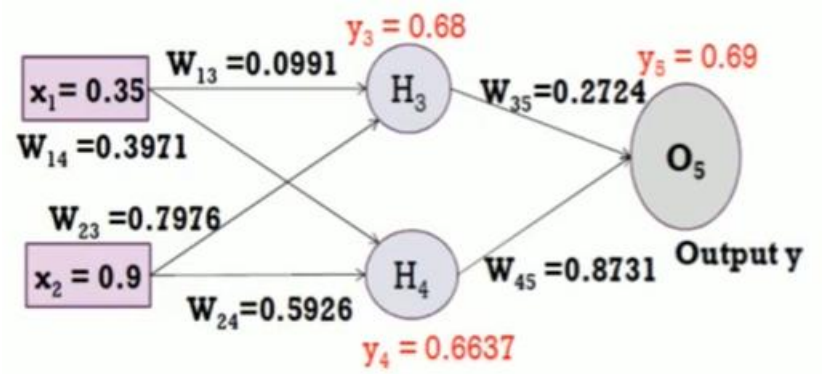
$$w_{45}\,(\text{new}) = \Delta w_{45} + w_{45}(\text{old}) = -0.0269 + (0.9) = 0.8731$$

$$\Delta w_{14} = \eta\, \delta_4\, x_1 = 1 * -0.0082 * 0.35 = -0.00287$$

$$w_{14}\,(\text{new}) = \Delta w_{14} + w_{14}(\text{old}) = -0.00287 + 0.4 = 0.3971$$

- Similarly, update all other weights

i	j	w_{ij}	δ_j	x_i	η	Updated w_{ij}
1	3	0.1	-0.00265	0.35	1	0.0991
2	3	0.8	-0.00265	0.9	1	0.7976
1	4	0.4	-0.0082	0.35	1	0.3971
2	4	0.6	-0.0082	0.9	1	0.5926
3	5	0.3	-0.0406	0.68	1	0.2724
4	5	0.9	-0.0406	0.6637	1	0.8731



- Forward Pass: Compute output for y3, y4 and y5.

$$a_j = \sum_j (w_{i,j} * x_i) \qquad yj = F(aj) = \frac{1}{1 + e^{-a_j}}$$

$a_1 = (w_{13} * x_1) + (w_{23} * x_2)$
$= (0.0991 * 0.35) + (0.7976 * 0.9) = 0.7525$
$y_3 = f(a_1) = 1/(1 + e^{-0.7525}) = 0.6797$

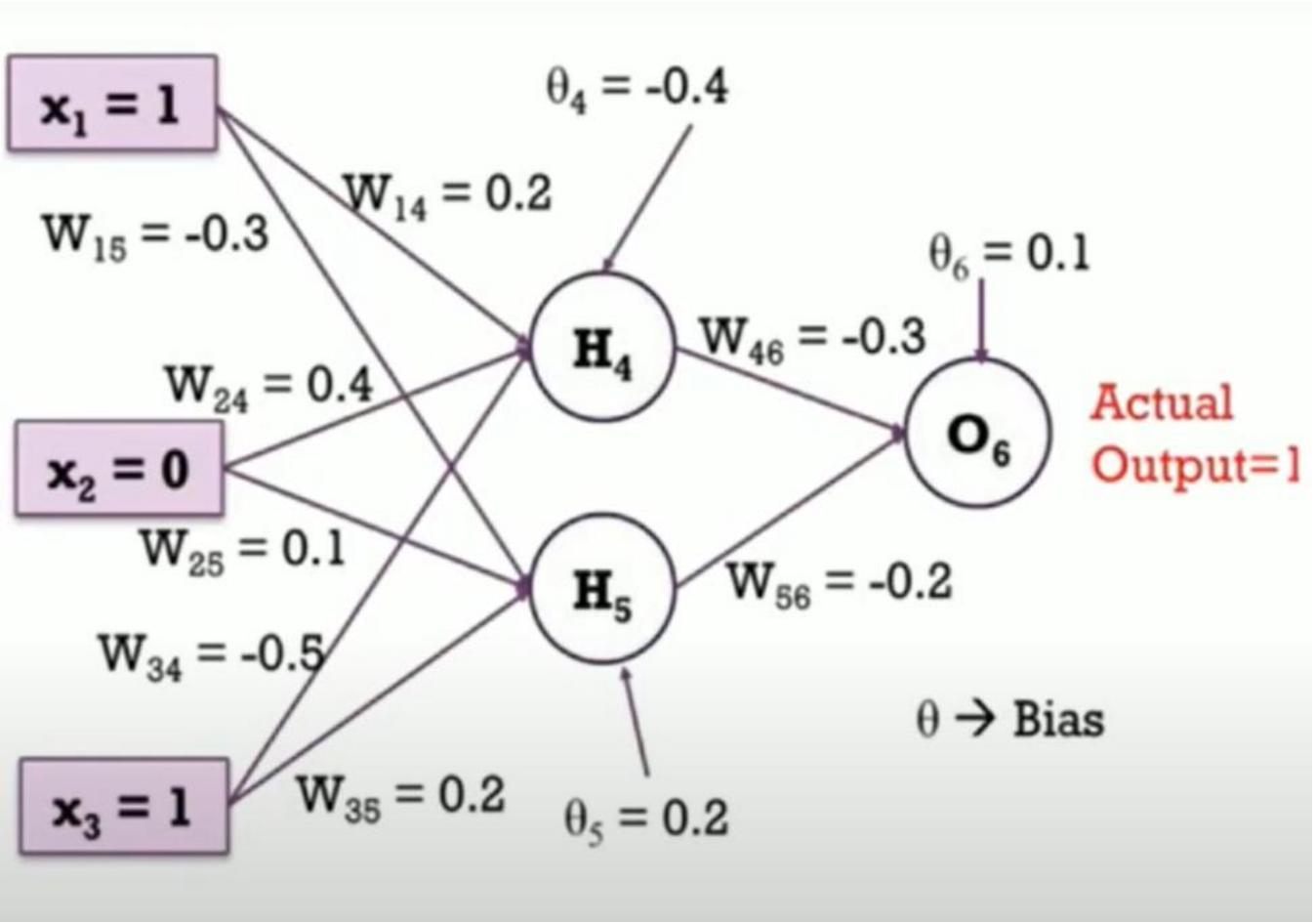
$a_2 = (w_{14} * x_1) + (w_{24} * x_2)$
$= (0.3971 * 0.35) + (0.5926 * 0.9) = 0.6723$
$y_4 = f(a_2) = 1/(1 + e^{-0.6723}) = 0.6620$

$a_3 = (w_{35} * y_3) + (w_{45} * y_4)$
$= (0.2724 * 0.6797) + (0.8731 * 0.6620) = 0.7631$
$y_5 = f(a_3) = 1/(1 + e^{-0.7631}) = \mathbf{0.6820}$ (Network Output)

$$\text{Error} = y_{target} - y_5 = -0.182$$

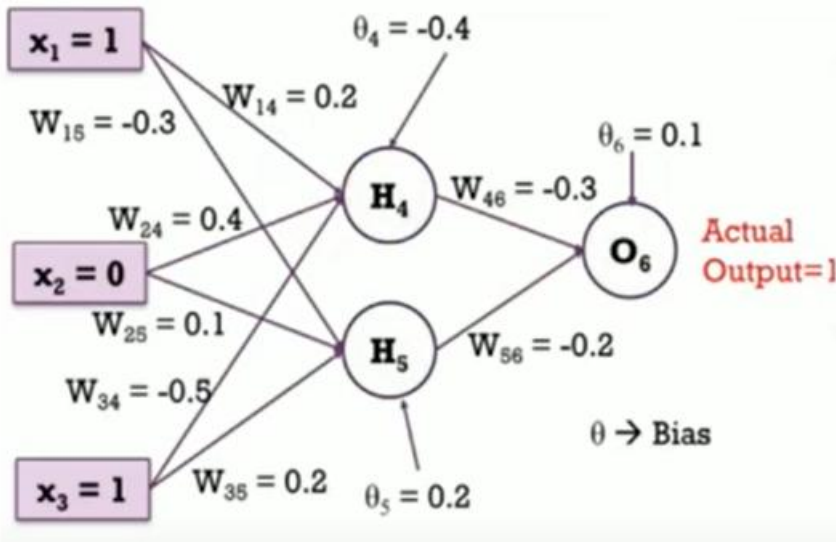
Example

Assume that the neurons have a sigmoid activation function, perform a forward pass and a backward pass on the network. Assume that the actual output y is 1 and the learning rate is 0.9. perform another forward pass



Forward pass:

Compute output for y4, y5, and y6



$$a_4 = (w_{14} * x_1) + (w_{24} * x_2) + (w_{34} * x_3) + \theta_4$$
$$= (0.2 * 1) + (0.4 * 0) + (-0.5 * 1) + (-0.4) = -0.7$$
$$O(H_4) = y_4 = f(a_4) = 1/(1 + e^{0.7}) = 0.332$$

$$a_5 = (w_{15} * x_1) + (w_{25} * x_2) + (w_{35} * x_3) + \theta_5$$
$$= (-0.3 * 1) + (0.1 * 0) + (0.2 * 1) + (0.2) = 0.1$$
$$O(H_5) = y_5 = f(a_5) = 1/(1 + e^{-0.1}) = 0.525$$

$$a_6 = (w_{46} * H_4) + (w_{56} * H_5) + \theta_6$$
$$= (-0.3 * 0.332) + (-0.2 * 0.525) + 0.1 = -0.105$$
$$O(O_6) = y_6 = f(a_6) = 1/(1 + e^{0.105}) = \mathbf{0.474}$$

$$\text{Error} = y_{target} - y_6 = 0.526$$

Backward Pass: Compute δ4, δ5 and δ6.

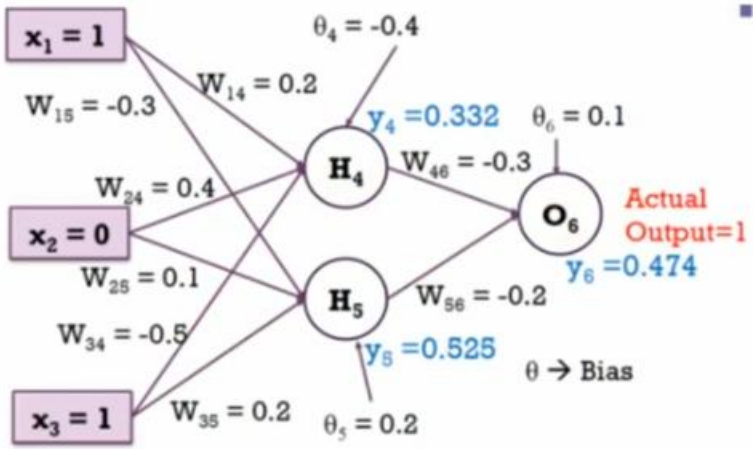
For output unit:

$$\delta_6 = y_6(1-y_6)\,(y_{target} - y_6)$$
$$= 0.474*(1-0.474)*(1-0.474) = 0.1311$$

For hidden unit:

$$\delta_5 = y_5(1-y_5)\, w_{56} * \delta_6$$
$$= 0.525*(1 - 0.525)*(-0.2 * 0.1311) = -0.0065$$

$$\delta_4 = y_4(1-y_4)\, w_{46} * \delta_6$$
$$= 0.332*(1 - 0.332)*(-0.3 * 0.1331) = -0.0087$$



Compute new wights

$$\Delta w_{ij} = \eta \delta_j o_i$$

$$\Delta w_{46} = \eta\, \delta_6\, y_4 = 0.9 * 0.1311 * 0.332 = 0.03917$$
$$w_{46}\,(new) = \Delta w_{46} + w_{46}(old) = 0.03917 + (-0.3) = -0.261$$

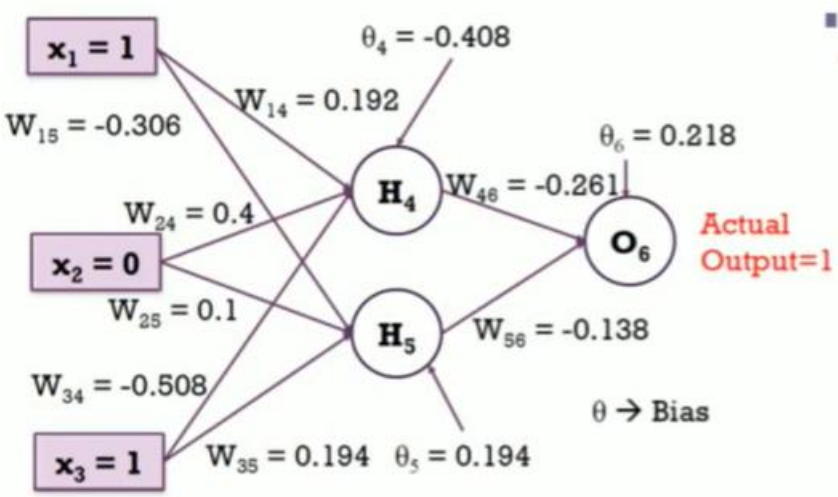
$$\Delta w_{14} = \eta\, \delta_4\, x_1 = 0.9 * -0.0087 * 1 = -0.0078$$
$$w_{14}\,(new) = \Delta w_{14} + w_{14}(old) = -0.0078 + 0.2 = 0.192$$

Similarly, update all other weights

i	j	w_{ij}	δ_j	x_i	η	Updated w_{ij}
4	6	-0.3	0.1311	0.332	0.9	-0.261
5	6	-0.2	0.1311	0.525	0.9	-0.138
1	4	0.2	-0.0087	1	0.9	0.192
1	5	-0.3	-0.0065	1	0.9	-0.306
2	4	0.4	-0.0087	0	0.9	0.4
2	5	0.1	-0.0065	0	0.9	0.1
3	4	-0.5	-0.0087	1	0.9	-0.508
3	5	0.2	-0.0065	1	0.9	0.194

Similarly, update bais weights

θ_j	Previous θ_j	δ_j	η	Updated θ_j
Θ_6	0.1	0.1311	0.9	0.218
Θ_5	0.2	-0.0065	0.9	0.194
Θ_4	-0.4	-0.0087	0.9	-0.408



Forward Pass: Compute output for y4, y5 and y6.

$a_4 = (w_{14} * x_1) + (w_{24} * x_2) + (w_{34} * x_3) + \theta_4$

$= (0.192 * 1) + (0.4 * 0) + (-0.508 * 1) + (-0.408) = -0.724$

$O(H_4) = y_4 = f(a_4) = 1/ (1 + e^{0.724}) = 0.327$

$a_5 = (w_{15} * x_1) + (w_{25} * x_2) + (w_{35} * x_3) + \theta_5$

$= (-0.306 * 1) + (0.1 * 0) + (0.194 * 1) + (0.194) = 0.082$

$O(H_5) = y_5 = f(a_5) = 1/ (1 + e^{-0.082}) = 0.520$

$a_6 = (w_{46} * H_4) + (w_{56} * H_5) + \theta_6$

$= (-0.261 * 0.327) + (-0.138 * 0.520) + 0.218 = 0.061$

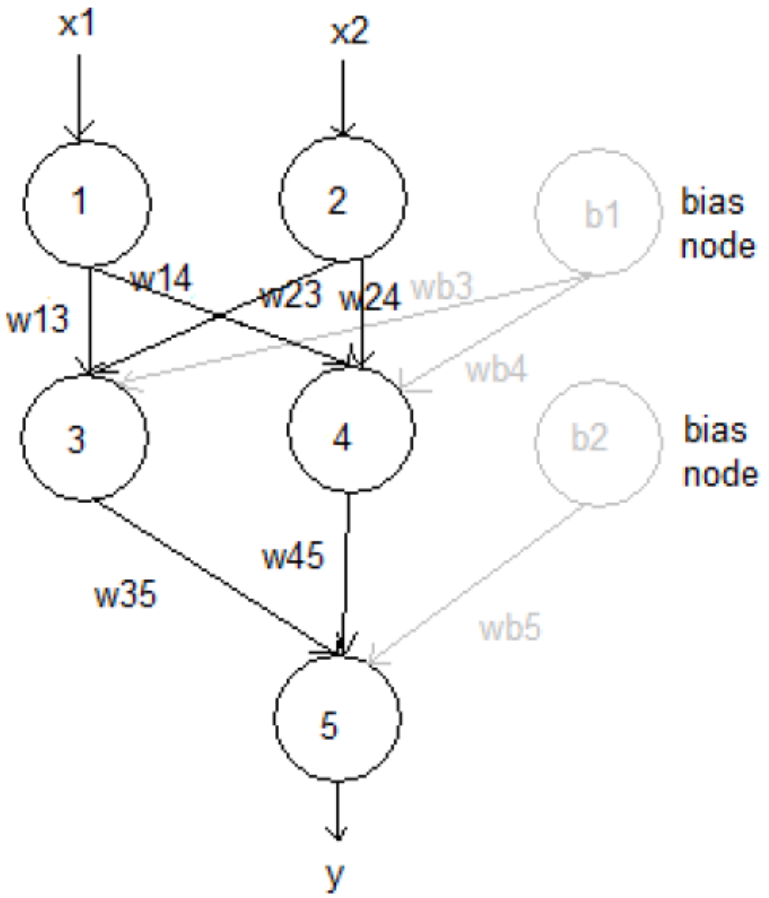
$O(O_6) = y_6 = f(a_6) = 1/ (1 + e^{-0.061}) = \mathbf{0.515}$ (Network Output)

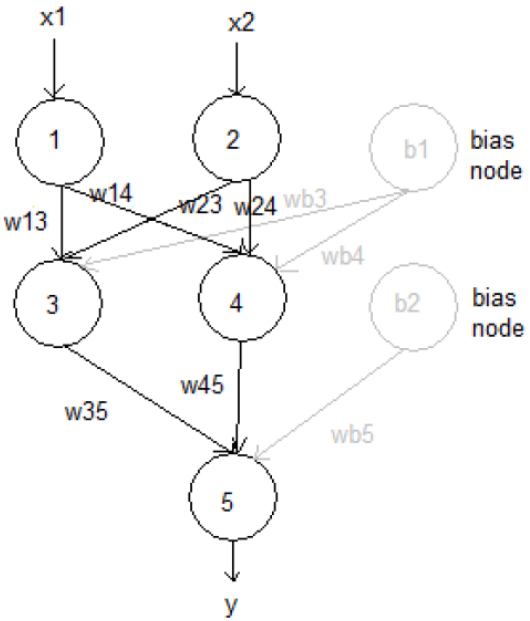
$$\text{Error} = y_{target} - y_6 = 0.485$$

Example

x1	x2	t
0	0	0
0	1	1
1	0	1
1	1	0

Assume learning rate = 0.3





Epoch: 1
Pattern: 1: x1 = 0, x2 = 0, t = 0

- Initial weights:

w13 = 0.3	w23 = -0.1	wb3 = 0.2
w14 = -0.2	w24 = 0.2	wb4 = -0.3
w35 = 0.4	w45 = -0.2	wb5 = 0.4

- Forward prop:
  - net3 = w13 * x1 + w23 * x2 + wb3 = 0.3 * 0 – 0.1 * 0 + 0.2 = 0.2
  - o3 = 1/(1 + e^-net3) = 1/(1 + e^-0.2) = 0.5498
  - net4 = w14 * x1 + w24 * x2 + wb4 = -0.2 * 0 + 0.2 * 0 – 0.3 = -0.3
  - o4 = 1/(1 + e^-net4) = 1/(1 + e^0.3) = 0.4256
  - net5 = w35 * o3 + w45 * o4 + wb5 = 0.4 * 0.5498 – 0.2 * 0.4256 + 0.4 = 0.5348
  - y = 1/(1+e^-net5) = 1/(1+e^-0.5348) = 0.6306
- Calculating error:
  - Err_p1 = 0.5 * (0 – 0.6306)^2 = 0.1988

Epoch: 1
Pattern: 1: x1 = 0, x2 = 0, t = 0

Back Prop:

1) Finding delta

$\delta 5 = y*(1 - y)*(t - y) = 0.6306 * (1 - 0.6306) * (0 - 0.6306) = - 0.1469$

$\delta 3 = o3(1 – o3)* \delta 5 * w35 = 0.5498 * (1 - 0.5498) * - 0.1469 * 0.4 = - 0.0145$

$\delta 4 = o4(1 – o4)* \delta 5 * w45 = 0.4256 * (1 - 0.4256) * - 0.1469 * -0.2 = 0.0072$

2) Finding new weights

$w35 := w35 + \eta * o3 * \delta 5 = 0.4 + 0.3 * 0.5498 * - 0.1469 = 0.3758$

$w45 := w45 + \eta * o4 * \delta 5 = -0.2 + 0.3 * 0.4256 * - 0.1469 = -0.2188$

$wb5 := wb5 + \eta * 1 * \delta 5 = 0.4 + 0.3 * 1 * - 0.1469 = 0.3559$

$w14 := w14 + \eta * x1 * \delta 4 = - 0.2 + 0.3 * 0 * 0.0072 = - 0.2$

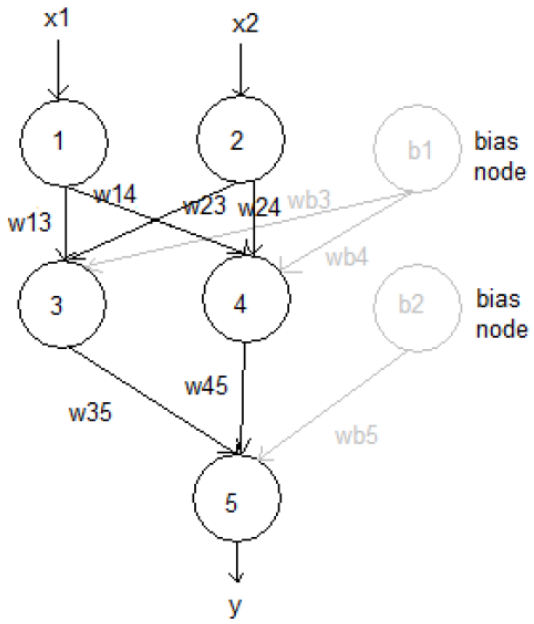
$w24 := w24 + \eta * x2 * \delta 4 = 0.2 + 0.3 * 0 * 0.0072 = 0.2$

$wb4 := wb4 + \eta * 1 * \delta 4 = - 0.3 + 0.3 * 1 * 0.0072 = - 0.2978$

$w13 := w13 + \eta * x1 * \delta 3 = 0.3 + 0.3 * 0 * - 0.0145 = 0.3$

$w23 := w23 + \eta * x2 * \delta 3 = - 0.1 + 0.3 * 0 * - 0.0145 = - 0.1$

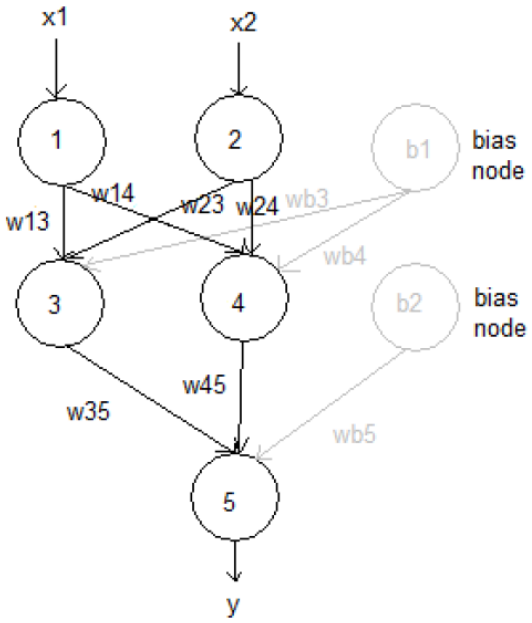
$wb3 := wb3 + \eta * 1 * \delta 3 = 0.2 + 0.3 * 1 * - 0.0145 = 0.1957$



Epoch: 1
Pattern: 2: x1 = 0, x2 = 1, t = 1

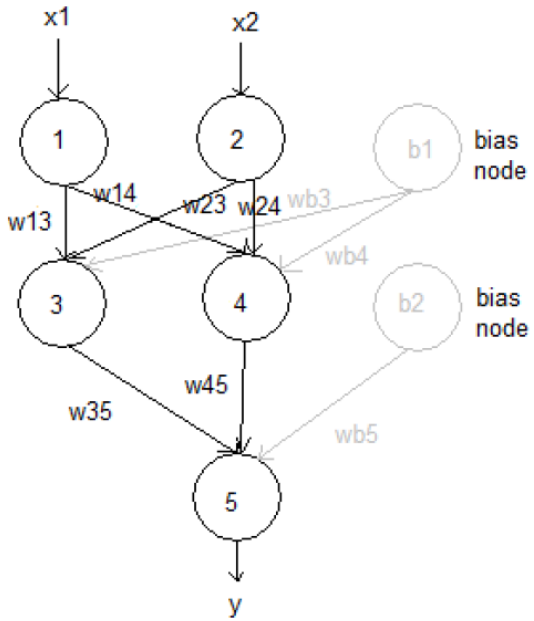
- weights:

w13 = 0.3	w23 = - 0.1	wb3 = 0.1957
w14 = - 0.2	w24 = 0.2	wb4 = - 0.2978
w35 = 0.3758	w45 = - 0.2188	wb5 = 0.3559



- Forward prop:
  - net3 = w13 * x1 + w23 * x2 + wb3 = …
  - o3 = 1/(1 + e^-net3) = …
  - net4 = w14 * x1 + w24 * x2 + wb4 = …
  - o4 = 1/(1 + e^-net4) = …
  - net5 = w35 * o3 + w45 * o4 + wb5 = …
  - y = 1/(1+e^-net5) = …
- Calculating error:
  - Err_p2 = 0.5 * (t – y)^2 = …

Epoch: 1
Pattern: 2: x1 = 0, x2 = 1, t = 1



Back Prop:

1) Finding delta

$\delta 5 = y*(1 - y)*(t - y) = \ldots$

$\delta 3 = o3(1 - o3)* \delta 5 * w35 = \ldots$

$\delta 4 = o4(1 - o4)* \delta 5 * w45 = \ldots$

2) Finding new weights

$w35 := w35 + \eta * o3 * \delta 5 = \ldots$

$w45 := w45 + \eta * o4 * \delta 5 = \ldots$

$wb5 := wb5 + \eta * 1 * \delta 5 = \ldots$

$w14 := w14 + \eta * x1 * \delta 4 = \ldots$

$w24 := w24 + \eta * x2 * \delta 4 = \ldots$

$wb4 := wb4 + \eta * 1 * \delta 4 = \ldots$

$w13 := w13 + \eta * x1 * \delta 3 = \ldots$

$w23 := w23 + \eta * x2 * \delta 3 = \ldots$

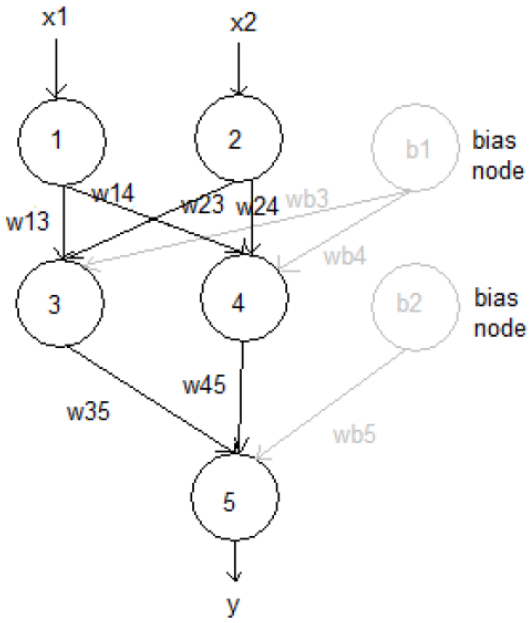
$wb3 := wb3 + \eta * 1 * \delta 3 = \ldots$

Epoch: 1
Pattern: 3: x1 = 1, x2 = 0, t = 1

- weights:

w13 = ?	w23 = ?	wb3 = ?
w14 = ?	w24 = ?	wb4 = ?
w35 = ?	w45 = ?	wb5 = ?

- Forward prop:
  - net3 = w13 * x1 + w23 * x2 + wb3 = …
  - o3 = 1/(1 + e^-net3) = …
  - net4 = w14 * x1 + w24 * x2 + wb4 = …
  - o4 = 1/(1 + e^-net4) = …
  - net5 = w35 * o3 + w45 * o4 + wb5 = …
  - y = 1/(1+e^-net5) = …
- Calculating error:
  - Err_p3 = 0.5 * (t – y)^2 = …



Epoch: 1
Pattern: 3: x1 = 1, x2 = 0, t = 1

Back Prop:

1) Finding delta

$\delta 5 = y*(1 - y)*(t - y) = \ldots$

$\delta 3 = o3(1 - o3)* \delta 5 * w35 = \ldots$

$\delta 4 = o4(1 - o4)* \delta 5 * w45 = \ldots$

2) Finding new weights

$w35 := w35 + \eta * o3 * \delta 5 = \ldots$

$w45 := w45 + \eta * o4 * \delta 5 = \ldots$

$wb5 := wb5 + \eta * 1 * \delta 5 = \ldots$

$w14 := w14 + \eta * x1 * \delta 4 = \ldots$

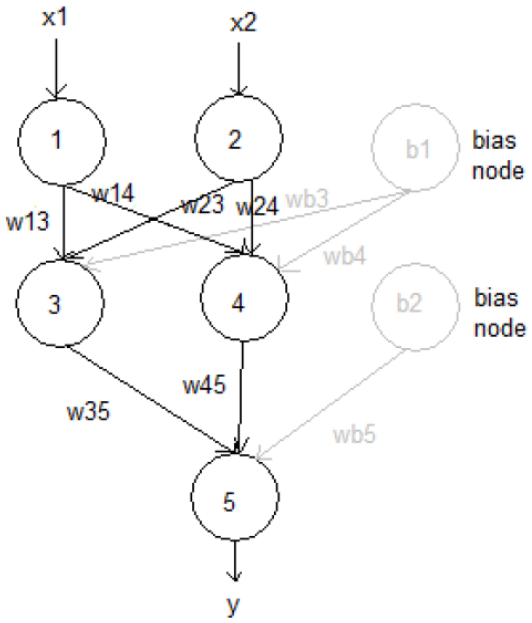
$w24 := w24 + \eta * x2 * \delta 4 = \ldots$

$wb4 := wb4 + \eta * 1 * \delta 4 = \ldots$

$w13 := w13 + \eta * x1 * \delta 3 = \ldots$

$w23 := w23 + \eta * x2 * \delta 3 = \ldots$

$wb3 := wb3 + \eta * 1 * \delta 3 = \ldots$

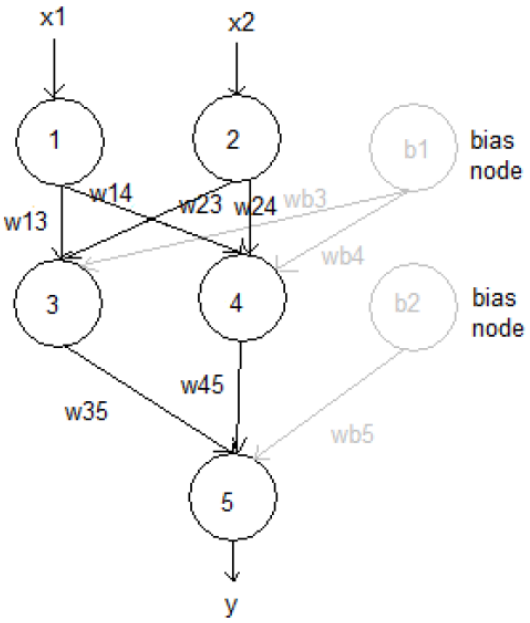


Epoch: 1
Pattern: 4: x1 = 1, x2 = 1, t = 0

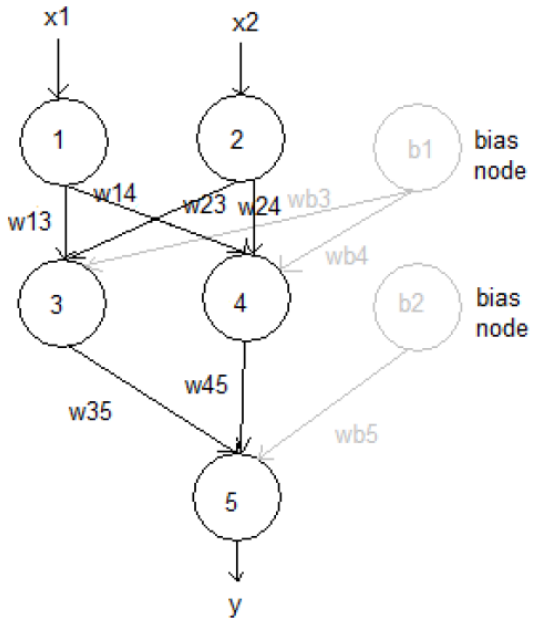
- weights:

w13 = ?	w23 = ?	wb3 = ?
w14 = ?	w24 = ?	wb4 = ?
w35 = ?	w45 = ?	wb5 = ?

- Forward prop:
  - net3 = w13 * x1 + w23 * x2 + wb3 = …
  - o3 = 1/(1 + e^-net3) = …
  - net4 = w14 * x1 + w24 * x2 + wb4 = …
  - o4 = 1/(1 + e^-net4) = …
  - net5 = w35 * o3 + w45 * o4 + wb5 = …
  - y = 1/(1+e^-net5) = …
- Calculating error:
  - Err_p4 = 0.5 * (t – y)^2 = …



Epoch: 1
Pattern: 4: x1 = 1, x2 = 1, t = 0



Back Prop:

1) Finding delta

$\delta 5 = y*(1 - y)*(t - y) = \ldots$

$\delta 3 = o3(1 - o3)* \delta 5 * w35 = \ldots$

$\delta 4 = o4(1 - o4)* \delta 5 * w45 = \ldots$

2) Finding new weights

$w35 := w35 + \eta * o3 * \delta 5 = \ldots$

$w45 := w45 + \eta * o4 * \delta 5 = \ldots$

$wb5 := wb5 + \eta * 1 * \delta 5 = \ldots$

$w14 := w14 + \eta * x1 * \delta 4 = \ldots$

$w24 := w24 + \eta * x2 * \delta 4 = \ldots$

$wb4 := wb4 + \eta * 1 * \delta 4 = \ldots$

$w13 := w13 + \eta * x1 * \delta 3 = \ldots$

$w23 := w23 + \eta * x2 * \delta 3 = \ldots$

$wb3 := wb3 + \eta * 1 * \delta 3 = \ldots$

End of Epoch 1

Total error = Err_p1 + Err_p2 + Err_p3 + Err_p4 = 0.1988 + …

If Total error <= tolerance (If given): Then stop training

If epoch number = max number of epochs (if given): Then stop training

Otherwise, run another epoch using last weights